

DDM501 Final Project Report

Real-Time Fraud Detection ML System

Repository: Final_Project_DMM501_Group1

Generated from repository implementation (as of 2026-04-14)

Contents

SECTION 1 — Problem Definition & Requirements	2
SECTION 2 — System Design & Architecture	4
SECTION 3 — Implementation	6
SECTION 4 — Testing & CI/CD	9
SECTION 5 — Responsible AI	10
SECTION 6 — Frontend Demo & UI/UX	10
SECTION 7 — Dataset Explanation (IMPORTANT)	11
SECTION 8 — Project Status & Verification	12
SECTION 9 — Risks & Limitations	13
SECTION 10 — Final Verdict	14
SECTION 11 — GUIDELINE FOR PRESENTATION	14
SECTION 12 — FINAL CHECKLIST	15
SECTION 13 — DEMO CHECKLIST (5 MIN LIVE)	17
Appendix — Key Figures (from artifacts/figures/)	19

SECTION 1 — Problem Definition & Requirements

1.1 Problem statement

Build an end-to-end ML system that scores credit-card transactions for fraud risk in near real time. The system must (1) train and benchmark models on the Kaggle Credit Card Fraud dataset, (2) serve predictions via an API, (3) provide a demo UI for live scoring, and (4) expose monitoring and alerting to support operational visibility.

1.2 Business context

Fraud scoring is used to reduce financial loss and operational cost by prioritizing transactions for review or intervention. In this project, the fraud score is treated as a decision-support signal with an explicit threshold policy (stored as model metadata and returned by the API).

1.3 Dataset description (Kaggle fraud dataset)

Dataset used in this repository: `data/archive/creditcard.csv`. The pipeline validates the schema and records it to `artifacts/reports/dataset_schema.json`.

- Rows/columns: 284,807 rows and 31 columns (Time, V1..V28, Amount, Class). See `artifacts/reports/dataset_schema.json`.
- Target: Class (0 = non-fraud, 1 = fraud).
- Feature contract used end-to-end (30 floats): [Time, V1..V28, Amount]. Stored in `artifacts/models/model_info.json` and enforced by the API.

1.4 Key challenges

- **Extreme class imbalance:** 492 fraud cases out of 284,807 total ($\approx 0.1727\%$). See `artifacts/reports/class_distribution.json`.
- **Accuracy is misleading:** A classifier can achieve $\approx 99.9\%$ accuracy while performing poorly on the fraud class; this repo prioritizes PR-AUC and threshold-tuned precision/recall.
- **Real-time serving constraints:** predictions must be fast, stable, and observable (latency and error rate monitored).
- **Operational policy:** threshold selection is a business decision (trade-off between false positives and false negatives) and must be explicit and versioned.

1.5 Success metrics (actual metrics and targets)

Note: Model results below are taken from generated artifacts in `artifacts/` (not fabricated). The benchmark workflow is implemented in `src/pipelines/run_model_workflow.py`.

Model metrics (from tuned test split) From `artifacts/reports/model_selection_summary.json`:

- **Baseline (Logistic Regression, tuned threshold 0.99):** PR-AUC=0.7929, ROC-AUC=0.9680, Precision=0.6742, Recall=0.8108, F1=0.7362.
- **Improved (LightGBM, tuned threshold 0.84):** PR-AUC=0.7161, ROC-AUC=0.8828, Precision=0.8852, Recall=0.7297, F1=0.8000.
- **Selected deployable model (by PR-AUC rule):** *Logistic Regression* (selected because improved PR-AUC < baseline PR-AUC). See `artifacts/models/model_info.json`.

System metrics (monitored) Implemented as Prometheus metrics in `src/monitoring/metrics.py`, exposed at GET `/metrics` by `src/api/main.py`:

- Request volume: `api_requests_total{endpoint,method,http_status}`
- Latency histogram: `api_request_latency_seconds_bucket{endpoint,method,...}`
- Prediction counts: `fraud_predictions_total{label}`
- Score aggregation: `fraud_scores_sum_total` and `fraud_scores_count_total`

Business metrics (demo-grade) This project treats PR-AUC as a proxy for review efficiency and uses threshold-tuned precision/recall to represent operational trade-offs (e.g., controlling false-positive review load while maintaining fraud recall).

1.6 Functional requirements (from implementation)

Aligned with `README.md` and verified in code:

- **F1:** POST `/predict` returns `fraud_probability` and threshold-based `fraud_label`. (`src/api/main.py`)
- **F2:** GET `/health` returns `model_loaded`, `model_version`, `expected_features`, `threshold`. (`src/api/main.py`)
- **F3:** GET `/metrics` exposes Prometheus metrics. (`src/api/main.py`, `src/monitoring/metrics.py`)
- **F4:** Training/benchmark workflow produces model artifacts + reports. (`src/pipelines/run_model_workflow.py`)
- **F5:** GET `/dataset/samples` returns dataset-backed samples for demo/verification. (`src/api/main.py`, `src/data/samples.py`)
- **F6:** Demo UI performs live scoring against backend `/predict`. (`frontend/`)
- **F7:** Observability stack configs: Prometheus + Grafana + alert rules via Docker Compose. (`deployment/`)

1.7 Non-functional requirements (from implementation)

- **Latency visibility and alerting:** Prometheus rule `FraudApiHighP95Latency` alerts if `p95 > 0.5s` for 10m. (`deployment/prometheus/alerts.yml`)
- **Reliability visibility:** Prometheus rule `FraudApiHigh5xxErrorRate` alerts if `5xx > 5%`. (`deployment/prometheus/alerts.yml`)
- **Reproducibility:** benchmark script writes deterministic split info and saves artifacts under `artifacts/`. (`artifacts/reports/split_info.json`)
- **CI quality gate:** GitHub Actions runs tests with coverage threshold $\geq 80\%$. (`.github/workflows/ci.yml`)
- **Containerized deployment:** API + frontend + MLflow + Prometheus + Grafana in Compose with healthchecks. (`deployment/docker-compose.yml`)

SECTION 2 — System Design & Architecture

2.1 High-level architecture (implemented)

Architecture is documented in `ARCHITECTURE.md` and implemented across `src/`, `frontend/`, and `deployment/`.

- **Training/Benchmarking:** `python -m src.pipelines.run_model_workflow` produces artifacts in `artifacts/`.
- **Model serving API:** FastAPI app in `src/api/main.py` loads `artifacts/models/final_model.joblib`.
- **Frontend demo:** static dashboard in `frontend/` streams transactions and calls `/predict`.
- **Monitoring:** Prometheus scrapes `/metrics`; Grafana dashboard auto-provisioned.
- **Experiment tracking:** MLflow SQLite backend at `artifacts/mlflow.db` (and Compose MLflow server).

2.2 Component breakdown

ML pipeline

- `src/pipelines/run_model_workflow.py`: end-to-end dataset validation, EDA, baseline vs improved model training, threshold tuning, SHAP, selection, and artifact export.
- `src/pipelines/train_pipeline.py`: smaller training pipeline (supports synthetic fallback; optional LightGBM if available).

API

- FastAPI endpoints in `src/api/main.py` with typed schemas in `src/api/schemas.py`.
- Model loading via `src/models/loader.py` using `MODEL_PATH` + metadata `model_info.json`.

Frontend

- Vanilla JS dashboard: `frontend/index.html`, `frontend/app.js`, `frontend/ui.js`.
- Streams transactions (real samples from `frontend/real-samples.json` or random generation) and scores them via `/predict`.

Monitoring

- Prometheus config: `deployment/prometheus/prometheus.yml` (scrape interval 5s).
- Alert rules: `deployment/prometheus/alerts.yml`.
- Grafana dashboard: `deployment/grafana/dashboards/fraud_api.json`.

2.3 Data flow (actual implementation)

- **Training:** `data/archive/creditcard.csv` → split (70/15/15) → baseline + LightGBM tuning → threshold sweep on validation → test evaluation → artifacts in `artifacts/`.
- **Inference:** frontend feature vector (30 floats) → `POST /predict` → `predict_proba` → label by stored threshold → UI + Prometheus metrics updated.
- **Observability:** `/metrics` scraped by Prometheus → Grafana panels + Prometheus alert evaluation.

2.4 Technology stack (from requirements.txt and deployment)

- ML: NumPy, Pandas, scikit-learn, LightGBM, SHAP, Matplotlib, Joblib
- Serving: FastAPI, Uvicorn, Pydantic
- Tracking: MLflow (SQLite backend in `artifacts/mlflow.db`)
- Monitoring: prometheus-client, Prometheus, Grafana
- Deployment: Docker, Docker Compose
- Frontend: HTML/CSS + vanilla JavaScript

2.5 Trade-offs (as implemented)

- **Accuracy vs. latency:** Simple tabular models (LogReg/LightGBM) keep inference fast and predictable; no deep model introduced.
- **Precision vs. recall:** Threshold tuning explicitly selects an operating point; the chosen threshold is stored in `artifacts/models/model_info.json`.
- **Model selection criterion:** This repo selects the deployable model primarily by **test PR-AUC** comparison between baseline and improved (see `src/pipelines/run_model_workflow.py`).
- **Demo realism:** Frontend simulates streaming with a loop and real dataset samples rather than integrating a message queue (Kafka/Kinesis not implemented).

SECTION 3 — Implementation

A. ML Pipeline

A1. Dataset usage

- Primary dataset path: `data/archive/creditcard.csv` (validated in `artifacts/reports/dataset_schema.json`).
- Feature columns used: 30 features `Time, V1..V28, Amount` (stored in `artifacts/models/model_info.json`).
- Split: `train=199,364, val=42,721, test=42,722` with `seed=42` (see `artifacts/reports/split_info.json`).

A2. Preprocessing

- API-side preprocessing is shape/typing only: `src/features/preprocess.py` converts list $\rightarrow$ `numpy.ndarray` of shape $(1, n)$.
- Baseline model uses a scikit-learn `Pipeline(StandardScaler, LogisticRegression)`; scaling is part of the artifact (see `src/pipelines/run_model_workflow.py`).
- Improved LightGBM uses raw numeric features; preprocessing recorded as identity in `artifacts/models/final_preprocessing_identity.json`.

A3. Models (baseline + improved)

- **Baseline:** Logistic Regression with `class_weight="balanced"` and `StandardScaler`. Artifact: `artifacts/models/baseline_logistic_regression_pipeline.joblib`.
- **Improved:** LightGBM (`LGBMClassifier`) tuned by validation PR-AUC across a small `ParameterGrid`. Artifact: `artifacts/models/improved_lightgbm.joblib`. Tuning log: `artifacts/benchmarks/improved_hyperparameter_tuning.csv`.
- **Final deployable:** Selected model exported to `artifacts/models/final_model.joblib` plus metadata `artifacts/models/model_info.json`.

A4. Evaluation metrics

Evaluation is implemented in `src/pipelines/run_model_workflow.py` and exported to artifacts:

- Confusion matrices: `artifacts/figures/confusion_matrix.png` (baseline) and `artifacts/figures/improved_confusion_matrix.png`.
- ROC/PR curves: `artifacts/figures/baseline_roc_curve.png`, `artifacts/figures/baseline_pr_curve.png`, `artifacts/figures/improved_roc_curve.png`, `artifacts/figures/improved_pr_curve.png`.
- Classification reports: `artifacts/reports/baseline_classification_report.json` and `artifacts/reports/improved_classification_report.json`.

A5. Threshold tuning

- Threshold sweep is computed on validation scores (`_threshold_sweep`) and the best threshold is chosen by max F1 (`_best_threshold_from_f1`).
- Threshold curves exported: `artifacts/benchmarks/baseline_threshold_tuning.csv` and `artifacts/benchmarks/improved_threshold_tuning.csv`.
- Tuned thresholds (from `artifacts/reports/model_selection_summary.json`): `baseline=0.99`, `improved=0.84`.

A6. Artifact outputs (actual files)

- Deployable model: `artifacts/models/final_model.joblib`
- Deploy metadata: `artifacts/models/model_info.json` (includes threshold, feature order, dataset path, metrics, selection timestamp)
- Benchmark summary: `artifacts/reports/benchmark_summary.md`
- SHAP outputs: `artifacts/figures/shap_summary.png`, `artifacts/benchmarks/shap_importance_table.csv`

B. Deployment

B1. FastAPI backend

Backend endpoint: `src/api/main.py`. The Docker image is defined in `deployment/api/Dockerfile` and the Compose service `api` in `deployment/docker-compose.yml`.

B2. Model loading logic

- At startup, the API attempts to load a model via `src/models/loader.py`.
- Primary mode: `MODEL_PATH` environment variable (Compose sets `/app/artifacts/models/final_model.joblib`).
- Metadata: the loader reads `model_info.json` next to the model artifact to obtain threshold, feature columns, dataset path, and selection timestamp.
- Safety: `/predict` returns HTTP 503 if model is not loaded; returns HTTP 422 for invalid feature length or non-finite values.

B3. API endpoints (actual contract)

Implemented in `src/api/main.py` with schemas in `src/api/schemas.py`:

- GET `/health`: status + model metadata (`model_loaded`, `model_version`, `expected_features`, `threshold`, `model_type`).
- POST `/predict`: request accepts `features: [float]` or `features_by_name: {name: float}`; response returns `fraud_probability`, `fraud_label`, `threshold`, `version`, request id.
- GET `/metrics`: Prometheus exposition format.

- GET `/dataset/samples`: samples from the recorded dataset path; supports `strategy=mixed|fraud|legit`. Additional demo-support endpoints exist: GET `/features/schema` and GET `/features/random`.

C. Frontend

C1. Real-time dashboard

- UI entry: `frontend/index.html` (Control Panel, Live Summary KPIs, Alert Panel, Chart, Live Transaction Feed).
- Logic: `frontend/app.js` orchestrates health polling and streaming scoring loop.
- Rendering: `frontend/ui.js` updates DOM, feed table, alert list, and chart canvas.

C2. Streaming logic (actual)

- Health poll every 5 seconds: GET `/health` to verify backend reachable and `expected_features==30`.
- Stream loop: on Start, the UI repeatedly generates a transaction and calls POST `/predict` at an interval of 0.5s / 1s / 2s.
- Data sources:
 - Real sample stream: dataset-backed sampling via GET `/dataset/samples` (production-like base rate, with rare fraud spikes for demo visibility).
 - Random generated stream: synthetic feature vectors produced by `frontend/demo-data.js`.
- Failure handling: stops stream after 4 consecutive errors (demo-friendly).

C3. Alert system and risk classification

Implemented in `frontend/app.js`:

- Risk tiers (policy-driven): *LOW* if score < `threshold_review`; *REVIEW* if score is between `threshold_review` and `threshold_high`; *HIGH* if score $\geq$ `threshold_high`.
- Alerts/cases are created for *REVIEW* and *HIGH* items and shown in the Alert Panel / Review queue.

D. Monitoring

D1. Prometheus metrics

Metrics are emitted in `src/monitoring/metrics.py` and exposed at GET `/metrics`.

D2. Grafana dashboards

Dashboard definition: `deployment/grafana/dashboards/fraud_api.json`. Panel titles include:

- API Requests Per Second
- Request Latency (95th percentile)
- Average Risk Score
- Decisions by Tier

D3. Alert rules

Alert rules are defined in `deployment/prometheus/alerts.yml` for:

- High 5xx error rate ($> 5\%$)
- High p95 latency ($> 0.5s$)
- Prediction rate too low / too high

SECTION 4 — Testing & CI/CD

4.1 Test types (from tests/)

- **Unit tests:** preprocessing and utility IDs (e.g., `tests/unit/test_preprocess.py`, `tests/unit/test_ids.py`).
- **Integration tests:** FastAPI endpoints with ASGI test client (e.g., `tests/integration/test_api_predict_happy_path.py`).
- **Data quality tests:** dataset loader behavior and target validity for synthetic data (`tests/data/test_data_quality_placeholder.py`).
- **Model validation tests:** training produces artifacts; loaded model outputs probabilities in range (`tests/model/test_model_validation_placeholder.py`).

4.2 Coverage strategy

CI enforces a coverage threshold via `pytest-cov`:

- Command: `pytest -q --cov=src --cov-report=term-missing --cov-fail-under=80`
- Workflow: `.github/workflows/ci.yml`

4.3 CI/CD pipeline (GitHub Actions)

- **CI:** installs dependencies, runs tests, fails if coverage $< 80\%$. (`.github/workflows/ci.yml`)
- **Docker validation:** builds API and frontend images; validates Compose file. (`.github/workflows/docker.yml`)

4.4 What is verified vs not verified (this report)

- **Verified by repository evidence:** implementation exists; artifacts contain real benchmark outputs; API endpoints and monitoring configs are present.
- **Not re-executed in this report generation:** CI execution, Docker builds, and live Prometheus/Grafana runtime behavior are not run here; treat them as *configured* and *reproducible* rather than proved live in this session.

SECTION 5 — Responsible AI

5.1 Explainability (SHAP)

- Implemented in `src/pipelines/run_model_workflow.py` for the LightGBM model.
- Outputs: `artifacts/figures/shap_summary.png` and `artifacts/benchmarks/shap_importance_table.csv`.

5.2 Fairness limitations

The dataset contains anonymized PCA components (V1..V28) and does not include protected attributes. As documented in `RESPONSIBLE_AI.md`, demographic fairness metrics cannot be computed; fairness discussion is limited to proxy slices (e.g., amount/time buckets) and should be treated as insufficient evidence rather than proof.

5.3 Privacy considerations

- The API contract accepts numeric features only (`PredictRequest` in `src/api/schemas.py`).
- Recommended logging stance (documented in `RESPONSIBLE_AI.md`): do not log raw feature vectors; log only request metadata and aggregated metrics.

5.4 Ethical risks and mitigations

- False positives: customer friction; mitigated via explicit threshold policy and a *REVIEW* tier in the UI (human-in-the-loop workflow).
- False negatives: fraud loss; mitigated by tracking recall and documenting trade-offs.
- Over-trusting explanations: SHAP is not causal; documented limitations in `RESPONSIBLE_AI.md`.

SECTION 6 — Frontend Demo & UI/UX

6.1 Dashboard layout (actual UI)

From `frontend/index.html`:

- Header: connection pill + model version + thresholds (review/high) + expected feature count.
- Control Panel: API Base URL, mode (real/random), speed, start/stop/reset.
- Live Summary KPIs: total processed, high risk (block), review queue, alert rate, avg risk score, last latency.
- Alert Panel: recent *REVIEW* and *HIGH* items.
- Chart: risk score over time with threshold overlays (review/high).
- Feed: append-only table with timestamp, request id, amount, score, tier, action, thresholds, latency, source.

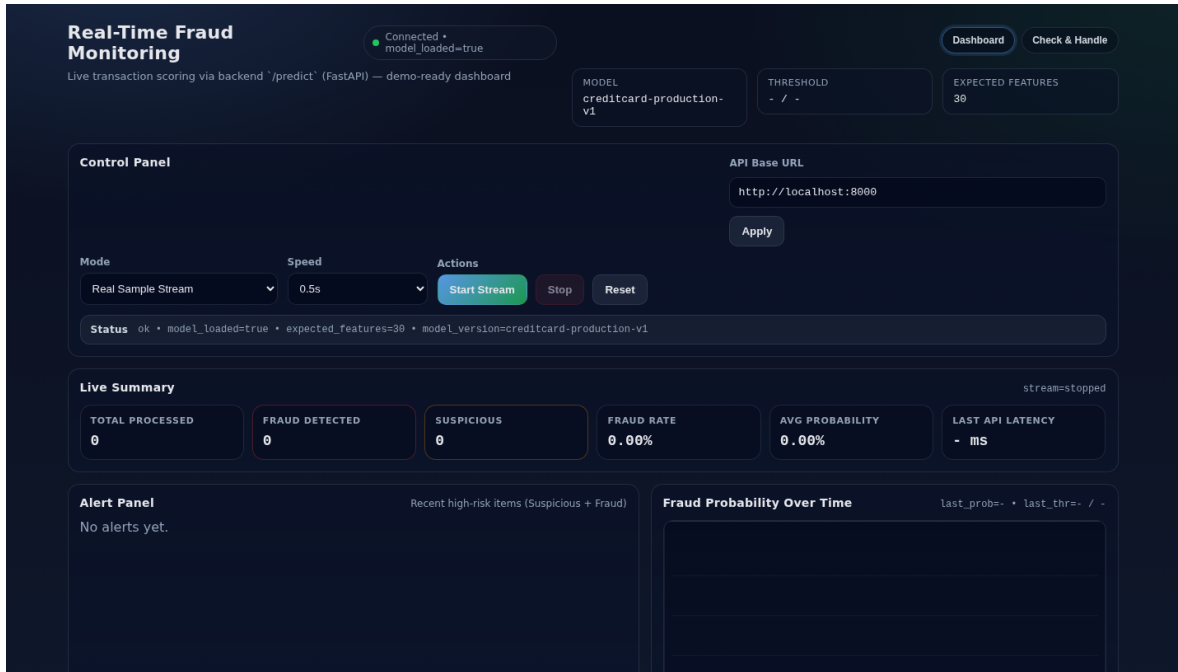


Figure 1: Frontend dashboard screenshot (connected to backend). Source: `artifacts/figures/frontend_dashboard_live.png`.

6.2 Transaction monitoring and abnormal detection

- Every generated transaction triggers a backend `/predict` call; the UI uses the returned `risk_score` as the primary signal.
- Risk tiering is policy-driven using backend thresholds (`threshold_review/threshold_high`) into LOW/REVIEW/HIGH.
- Alerts are UI-only (no external notification channel implemented).

6.3 Streaming demo

Streaming is simulated (no Kafka/WebSockets). The demo intentionally keeps architecture simple:

- A loop generates transactions (`frontend/demo-data.js`) and calls `/predict` at a fixed interval.
- The UI stops the stream after repeated errors to keep the live presentation stable.

SECTION 7 — Dataset Explanation (IMPORTANT)

7.1 Dataset structure

From `artifacts/reports/dataset_schema.json`:

- 31 columns: Time, V1..V28, Amount, Class.
- Training/inference feature vector uses 30 columns (all except `Class`).

7.2 Imbalance issue ($\approx 0.17\%$ fraud)

From `artifacts/reports/class_distribution.json`:

- Non-fraud (0): 284,315 (ratio=0.9982725144; $\approx 99.8273\%$)
- Fraud (1): 492 (ratio=0.0017274856; $\approx 0.1727\%$)

7.3 Why accuracy is misleading

The baseline classification report (`artifacts/reports/baseline_classification_report.json`) reports accuracy ≈ 0.99899 . This looks excellent, but is dominated by the majority class. Fraud detection performance must be judged with:

- **PR-AUC:** focuses on performance under imbalance.
- **Precision/recall at operating threshold:** reflects review load vs missed fraud trade-off.

7.4 Why recall/threshold matter

This system explicitly chooses and stores a threshold:

- Threshold sweep and best F1 threshold are computed on validation data.
- The threshold is stored in `artifacts/models/model_info.json` and returned by `/predict`.
- Frontend uses the threshold from the backend response as the source of truth for risk labeling.

7.5 How the dataset is used in training vs demo

- **Training/benchmarking:** `src/pipelines/run_model_workflow.py` reads `data/archive/creditcard.csv` and generates all reports/figures/models.
- **Demo streaming (frontend):** `frontend/real-samples.json` contains pre-extracted feature vectors (and optional `class_label`) from the same dataset path for stable UI streaming (240 samples; 60 fraud-labeled).
- **Demo/verification (API):** `GET /dataset/samples` samples rows directly from the recorded dataset path in model metadata (useful for verification scripts such as `verify_system.py`).

SECTION 8 — Project Status & Verification

8.1 Status rubric

- **Verified Working:** verified by automated tests in this repo and/or by generated artifacts with internal validation checks.
- **Partial:** implemented but missing pieces for full demo/production expectation.
- **Unverified:** implemented/configured, but not proven running in this report generation.
- **Missing:** not implemented in this repository.

8.2 Component-by-component status (based on repo evidence)

Component	Status	Evidence / Notes
ML pipeline (benchmark)	Verified Working	Outputs exist in <code>artifacts/</code> including schema, split, metrics, figures, SHAP, and <code>final_model.joblib</code> ; validation checks stored in <code>artifacts/reports/model_validation_checks.json</code> .
ML pipeline (synthetic training)	Verified Working	<code>src/pipelines/train_pipeline.py</code> is exercised by tests in <code>tests/model/test_model_validation_placeholder.py</code> .
API	Unverified	Implemented in <code>src/api/main.py</code> with tests under <code>(/health,/predict,/metrics,/dataset/samples)</code> ; integration tests under <code>tests/integration/</code> ; runtime not launched in this report generation.
Frontend dashboard	Unverified	Implemented in <code>frontend/</code> ; not launched in this report generation.
Docker (images + Compose)	Unverified	<code>deployment/api/Dockerfile</code> , <code>deployment/frontend/Dockerfile</code> , <code>deployment/docker-compose.yml</code> exist; GitHub Actions contains a Docker build workflow.
Monitoring (Prometheus/Grafana)	Unverified	Prometheus + Grafana configs exist (<code>deployment/prometheus/*</code> , <code>deployment/grafana/*</code>); requires running Compose to observe live panels.
CI/CD (GitHub Actions)	Unverified	Workflows exist in <code>.github/workflows/</code> . This report does not access GitHub to confirm latest run status.

8.3 Consistency note (important)

The authoritative deployed threshold and selected model are taken from `artifacts/models/model_info.json` (selection timestamp 2026-04-14T08:17:58.395658+00:00). Some older/internal narrative documents in the repo may reference different model paths or thresholds; for demo correctness, follow the artifact metadata used by the API loader.

SECTION 9 — Risks & Limitations

9.1 Technical risks

- Feature order mismatch risk: the API validates *length* and numeric finiteness, but cannot prove semantic ordering unless `features_by_name` is used with metadata.
- Dependency risk: LightGBM/SHAP can be heavier dependencies; however the selected deployable artifact is Logistic Regression pipeline in this artifact set.
- No auth/rate limiting: the API is demo-grade (documented as future work in `ARCHITECTURE.md`).

9.2 Dataset limitations

- No protected attributes: fairness analysis is limited.
- Dataset is static and historical; concept drift not modeled.
- PCA components reduce interpretability; SHAP provides relative importance but not business-meaningful feature names.

9.3 System limitations

- “Real-time” in demo is request/response scoring plus UI streaming simulation (no queue or event bus).
- Monitoring observes service metrics (latency, errors, counts) but does not implement drift detection or automated retraining.

SECTION 10 — Final Verdict

10.1 Is the system end-to-end?

Yes, at demo scope: this repository contains (1) a full benchmark/training pipeline producing deployable artifacts, (2) a serving API that loads those artifacts and exposes required endpoints, (3) a frontend that performs live scoring against the API, and (4) monitoring/alerting configuration via Prometheus and Grafana.

10.2 Is it demo-ready?

Likely demo-ready with a local run: the repo provides a full run guide (`QUICK_START.md`) and a verification script (`verify_system.py`). This report does not execute the stack; “demo-ready” should be confirmed by running the demo checklist in Section 13.

10.3 What is missing for production-level readiness?

- Authentication, authorization, rate limiting, and secure logging/redaction.
- Drift detection, automated retraining, and model rollout controls (canary/A-B).
- Persistent external artifact store and model registry (beyond local `artifacts/`).
- Data governance and robust fairness evaluation (requires additional attributes or approved proxies).

SECTION 11 — GUIDELINE FOR PRESENTATION

11.1 Step-by-step talk track (slide-friendly)

1. **Problem framing (30s):** fraud detection with extreme imbalance; goal is near-real-time scoring + operational visibility.
2. **Dataset (45s):** show schema and imbalance numbers from `artifacts/reports/dataset_schema.json` and `artifacts/reports/class_distribution.json`.

3. **Pipeline (60s):** explain `src/pipelines/run_model_workflow.py` outputs: EDA, split, baseline vs LightGBM, threshold tuning, model selection, SHAP artifacts.
4. **Model results (45s):** present tuned metrics table (PR-AUC, precision, recall, F1) and explain threshold trade-offs; emphasize PR-AUC for imbalance.
5. **Serving API (60s):** demo `/health` then `/predict`; highlight validation (feature length, numeric checks) and metadata in responses.
6. **Monitoring (45s):** open `/metrics`; explain what Prometheus scrapes and what Grafana shows; mention alert rules.
7. **Frontend demo (60–90s):** start stream; narrate KPIs, feed, alerts, and chart; explain LOW/REVIEW/HIGH policy with thresholds.
8. **Responsible AI (30–45s):** explain SHAP usage and fairness/privacy limitations from `RESPONSIBLE_AI.md`.

11.2 What to emphasize (rubric-aligned)

- **Dataset + imbalance:** why PR-AUC and recall/precision matter more than accuracy.
- **System reasoning:** explicit threshold policy stored and returned; observability built in.
- **Demo realism:** UI uses backend predictions (not mocked).
- **Reproducibility:** artifacts in `artifacts/` and workflows in `.github/workflows/`.

SECTION 12 — FINAL CHECKLIST

Legend: [x] = confirmed by repository evidence; [] = requires live verification or is not confirmed here.

A. Problem & Requirements

- [x] Problem statement and business context documented (`README.md`, this report).
- [x] Dataset schema and imbalance documented from real artifacts (`artifacts/reports/*`).
- [x] Functional requirements mapped to endpoints and modules (`src/api/*`).
- [] Live run verifies success metrics during demo (latency/error behavior).

B. Architecture

- [x] High-level architecture diagram and flows documented (`ARCHITECTURE.md`).
- [x] Data flow (train → artifacts → serve) implemented.
- [] Full stack validated end-to-end via Docker Compose on target demo machine.

C. ML Pipeline

- [x] Benchmark workflow produces reports/figures/models (`src/pipelines/run_model_workflow.py`, `artifacts/`).
- [x] Threshold tuning implemented and exported (`artifacts/benchmarks/*threshold*`).
- [x] SHAP explainability artifacts generated (`artifacts/figures/shap_summary.png`).
- [] Re-run pipeline on demo machine and confirm identical outputs (or record new model version).

D. API

- [x] Endpoints implemented: `/health`, `/predict`, `/metrics`, `/dataset/samples`.
- [x] Input validation for length + numeric values.
- [] Live API started and validated via curl/Swagger on demo machine.

E. Frontend

- [x] Dashboard implemented and calls backend `/predict` (`frontend/`).
- [] Live UI verified in browser with running backend.

F. Monitoring

- [x] Prometheus + Grafana configs and dashboard exist (`deployment/`).
- [x] Alert rules defined (`deployment/prometheus/alerts.yml`).
- [] Prometheus target up and Grafana dashboard shows live metrics during demo.

G. Testing

- [x] Unit/integration/model/data tests exist (`tests/`).
- [] Tests executed locally on demo machine with coverage $\geq 80\%$.

H. CI/CD

- [x] GitHub Actions CI workflow exists (`.github/workflows/ci.yml`).
- [x] Docker build/compose validation workflow exists (`.github/workflows/docker.yml`).
- [] Confirm latest CI runs are green before submission (on GitHub).

I. Documentation

- [x] Runbook exists (`QUICK_START.md`).
- [x] Responsible AI doc exists (`RESPONSIBLE_AI.md`).
- [x] This submission report matches implementation and artifact values.

J. Demo

- [] Live demo rehearsed end-to-end (Section 13).
- [] Backup plan prepared (screenshots of Grafana, saved artifacts, and Swagger screenshots).

SECTION 13 — DEMO CHECKLIST (5 MIN LIVE)

5-minute demo flow (step-by-step)

1. Start API (Terminal 1):

- Run the model workflow if artifacts are missing: `python -m src.pipelines.run_model_workflow --data-path data/archive/creditcard.csv --artifacts-root artifacts --seed 42`
- Start API (explicit model path): `MODEL_PATH=artifacts/models/final_model.joblib FRAUD_THRESHOLD=0.99 MODEL_VERSION=creditcard-production-v1 uvicorn src.api.main:app --host 0.0.0.0 --port 8000`

2. Quick API checks (browser or curl):

- Open Swagger: <http://localhost:8000/docs>
- Confirm health: <http://localhost:8000/health> shows `model_loaded=true` and `expected_features=30`.
- Open metrics: <http://localhost:8000/metrics>.

3. Open frontend (Terminal 2 + browser):

- `cd frontend && python -m http.server 8080 --bind 127.0.0.1`
- Open: <http://127.0.0.1:8080/index.html>

4. Run stream:

- Ensure connection pill is green.
- Mode = *Real Sample Stream* and Speed = 1s.
- Click *Start Stream*.

5. Show fraud detection and alerts:

- Point to `risk_score` values updating in the feed.
- Show Alert Panel entries for *REVIEW* / *HIGH*.

6. Show chart:

- Point out the rolling score chart and the threshold lines (review/high).

7. Show monitoring quickly (optional):

- If running Docker Compose: open Grafana <http://localhost:3000> and show the Fraud API dashboard.
- If not: open `/metrics` and show counters increasing.

8. **Stop stream:**

- Click *Stop* and confirm KPIs freeze.

Appendix — Key Figures (from artifacts/figures/)

This appendix expands visual evidence coverage for dataset properties, model behavior, threshold policy selection, frontend operation, and deployment/monitoring runtime.

A. Dataset and EDA visuals

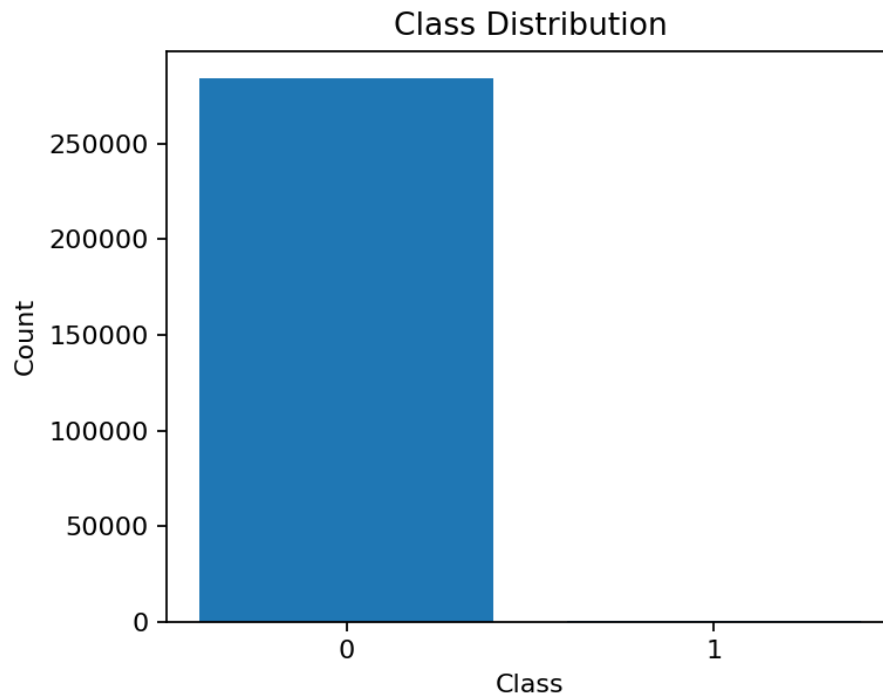


Figure 2: Class distribution (fraud vs non-fraud). Source: `artifacts/figures/class-distribution.png`

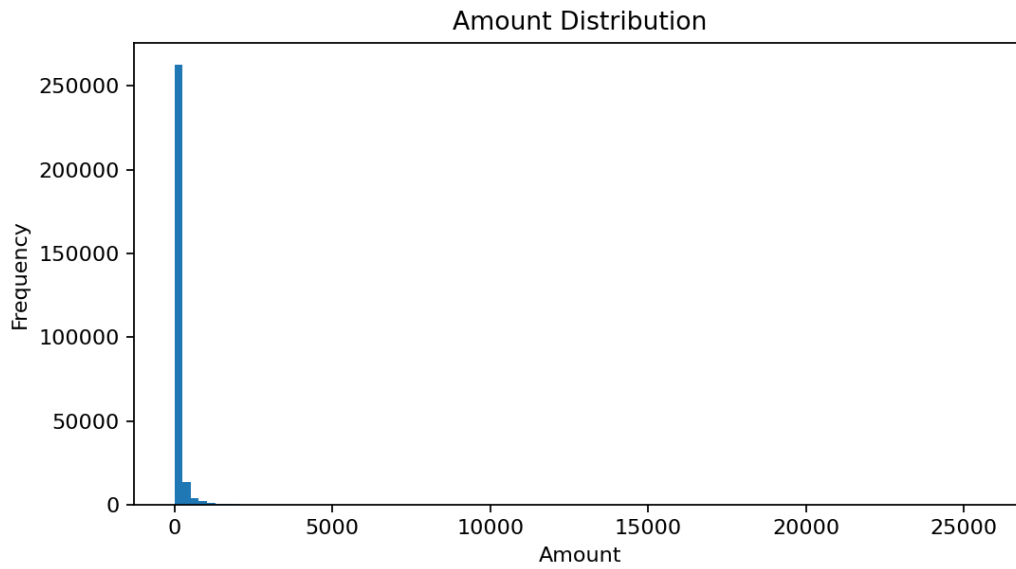


Figure 3: Transaction amount distribution. Source: `artifacts/figures/amount_distribution.png`

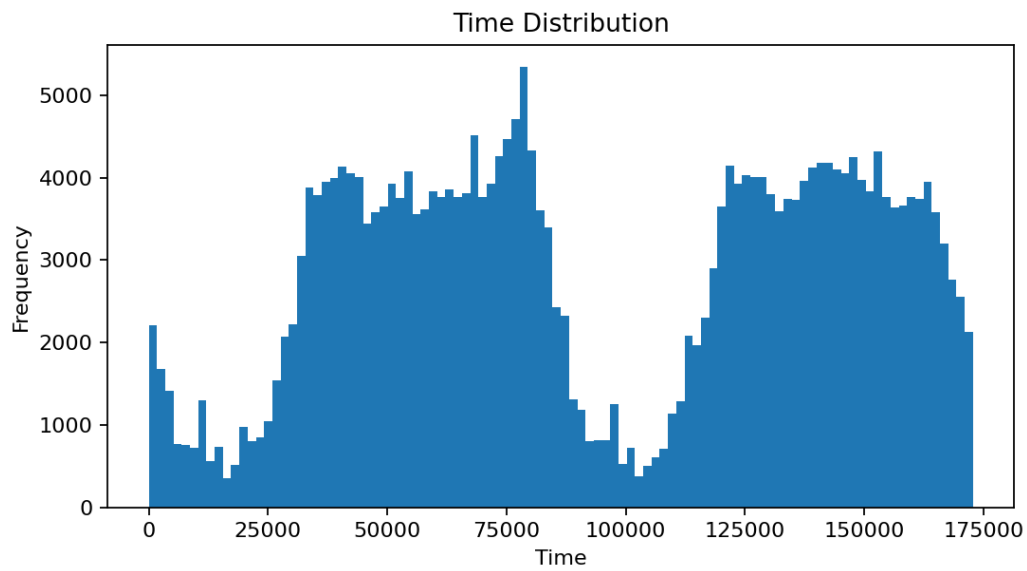


Figure 4: Transaction time distribution. Source: `artifacts/figures/time_distribution.png`

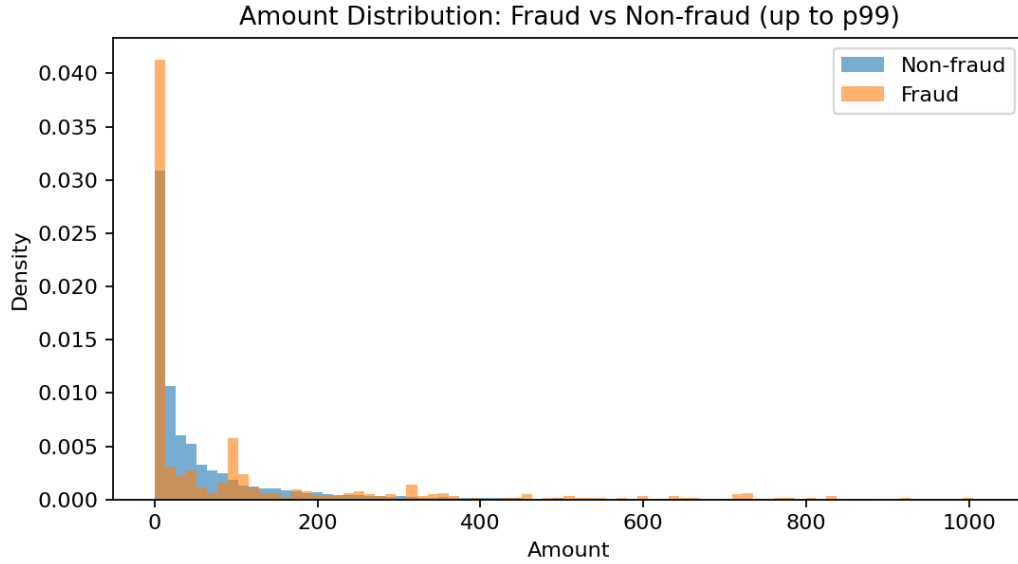


Figure 5: Fraud vs non-fraud amount comparison. Source: `artifacts/figures/fraud_vs_-nonfraud_amount.png`

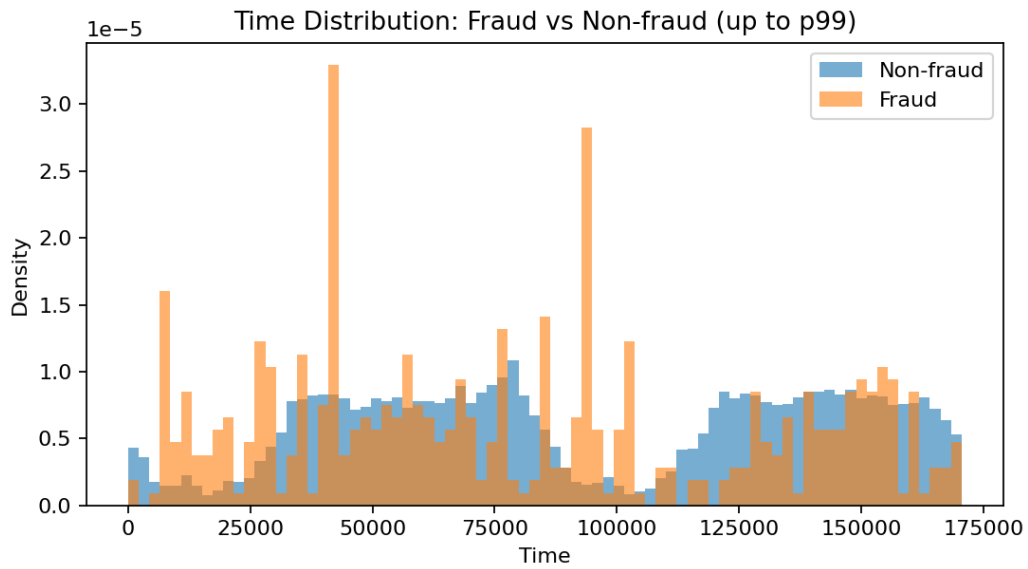


Figure 6: Fraud vs non-fraud time comparison. Source: `artifacts/figures/fraud_vs_-nonfraud_time.png`

B. Model evaluation and selection visuals

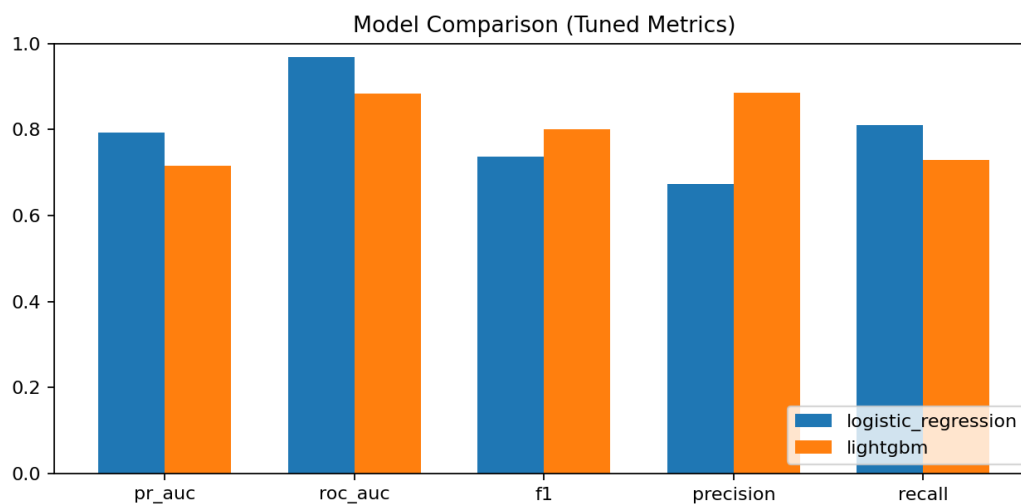


Figure 7: Model comparison of tuned metrics. Source: `artifacts/figures/model_comparison.png`

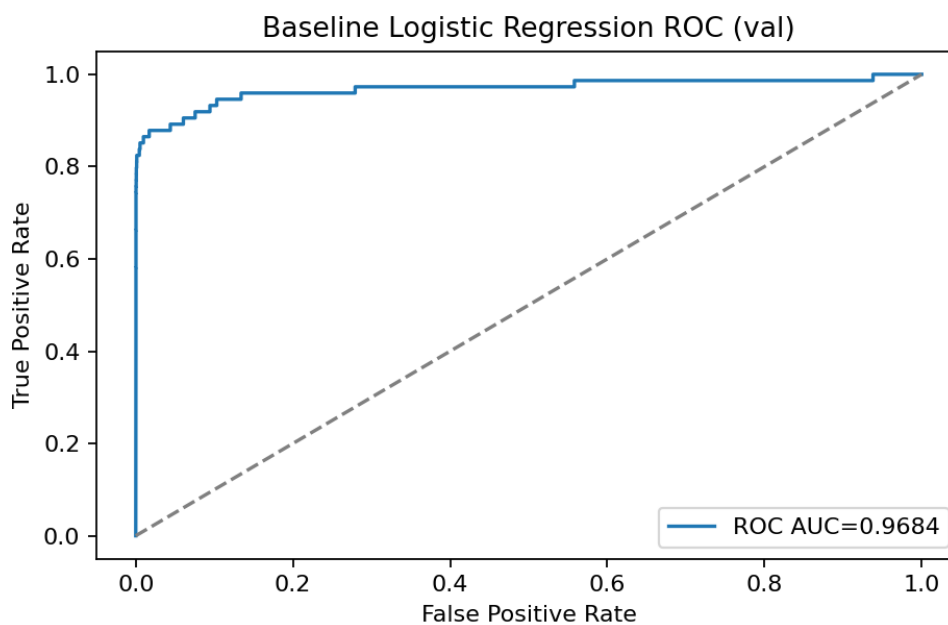


Figure 8: Baseline ROC curve. Source: `artifacts/figures/baseline_roc_curve.png`

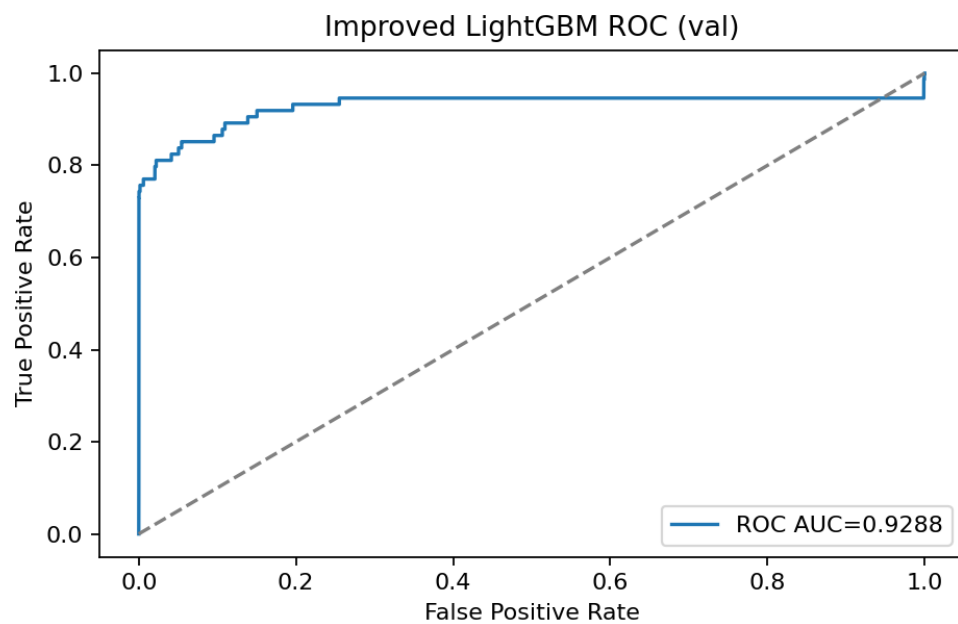


Figure 9: Improved ROC curve. Source: `artifacts/figures/improved_roc_curve.png`

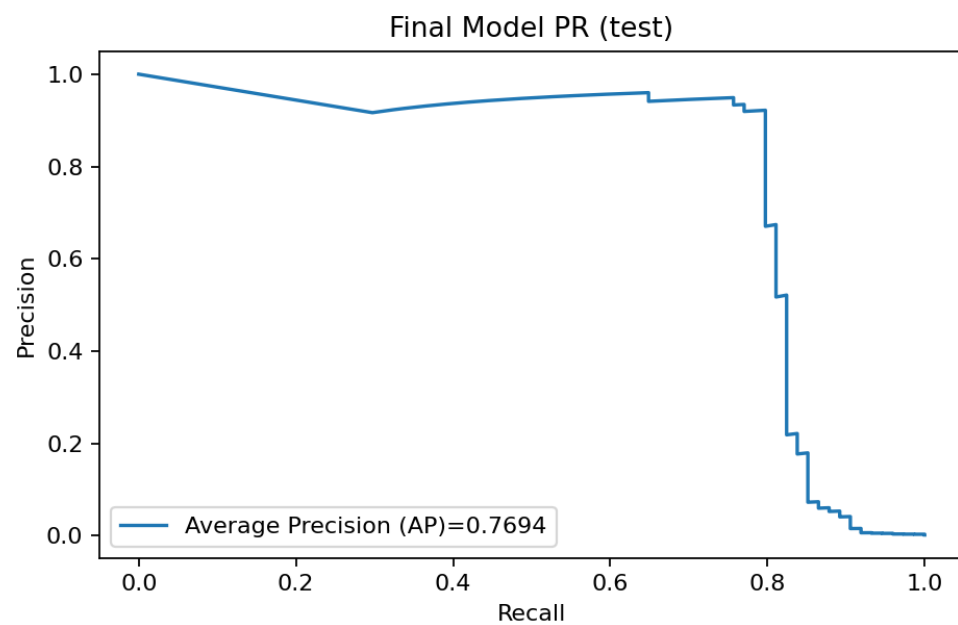


Figure 10: Final selected model PR curve. Source: `artifacts/figures/final_pr_curve.png`

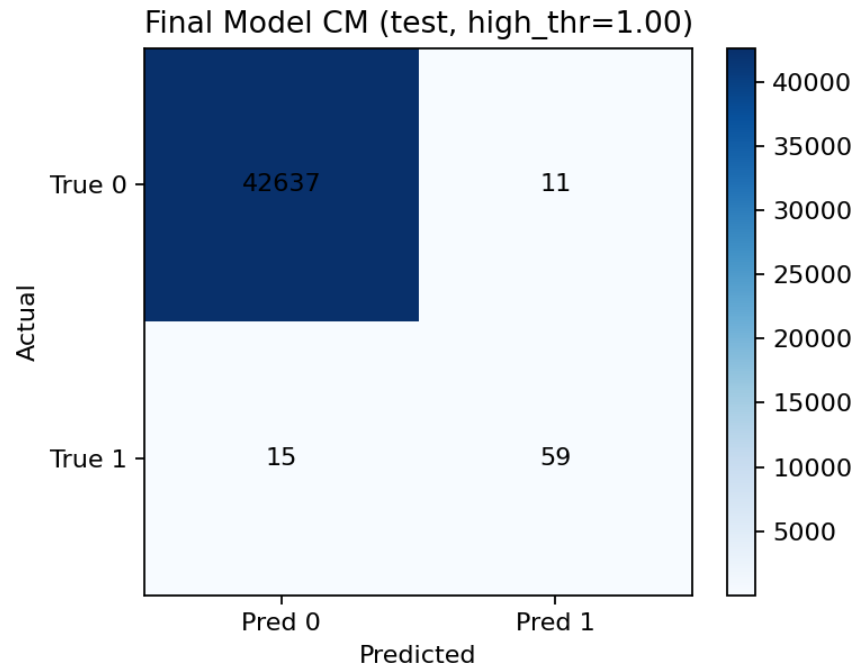


Figure 11: Final selected model confusion matrix. Source: `artifacts/figures/final_confusion_matrix.png`

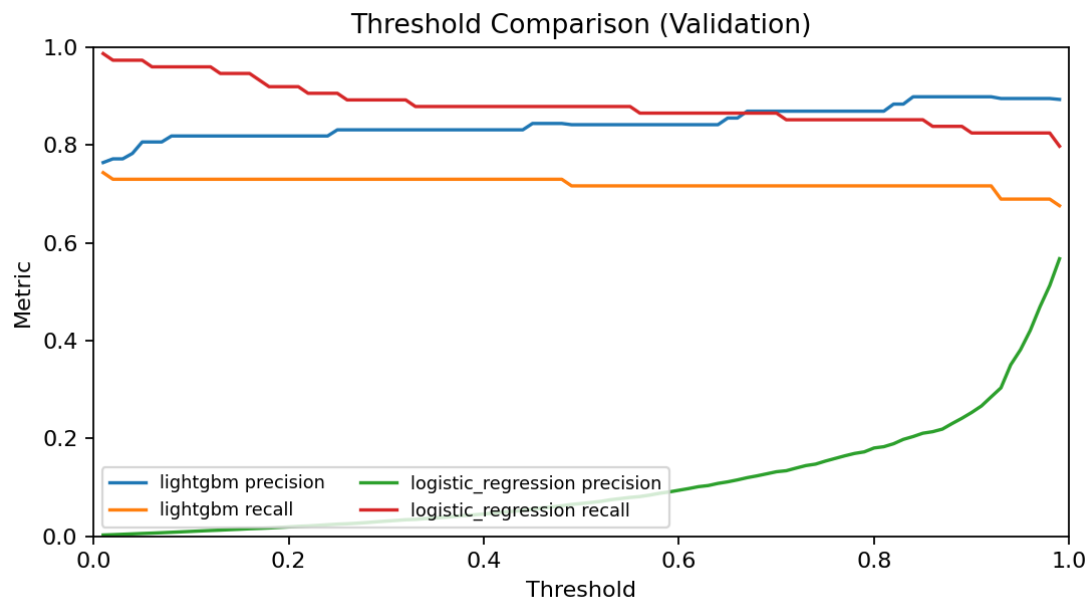


Figure 12: Threshold comparison curves. Source: `artifacts/figures/threshold_comparison.png`

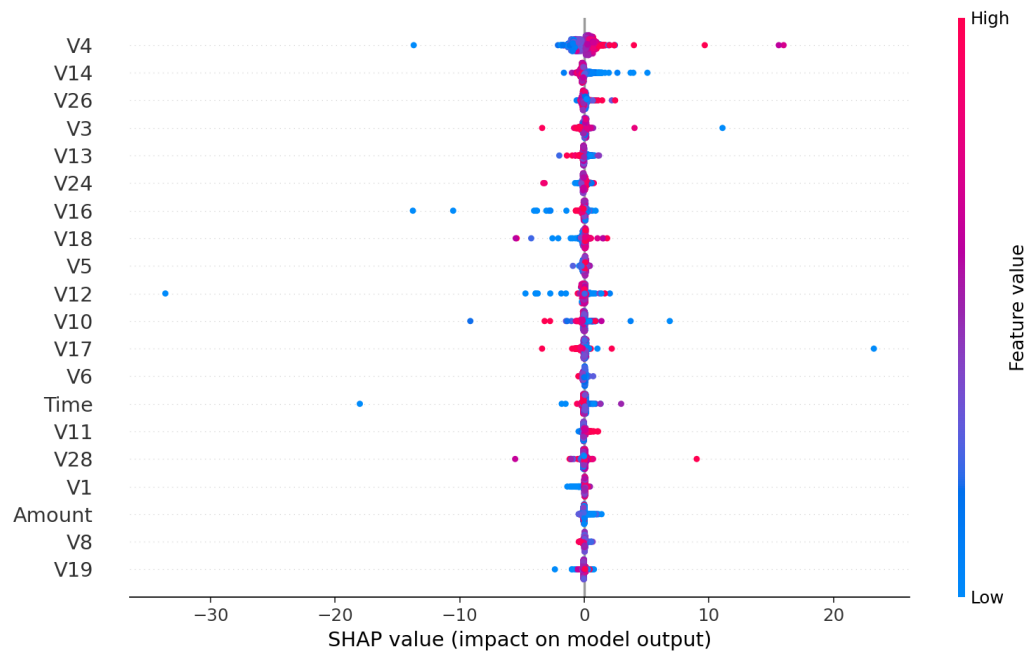


Figure 13: SHAP summary plot (LightGBM explainability). Source: `artifacts/figures/shap_summary.png`

C. Frontend and deployment visuals

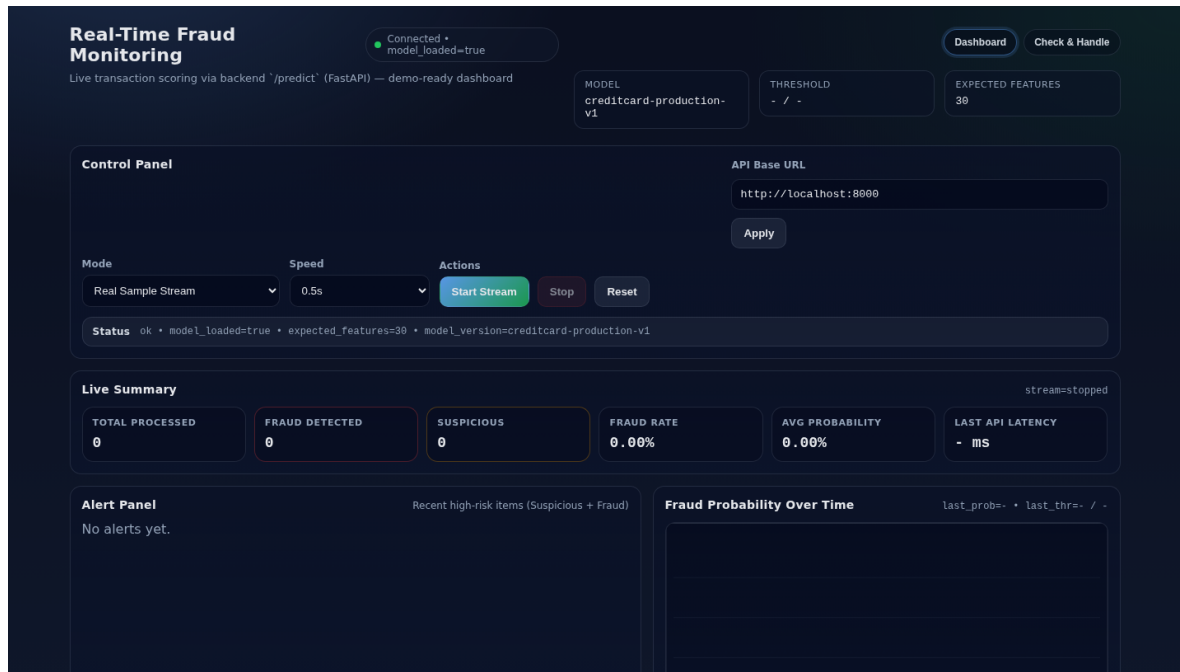


Figure 14: Frontend dashboard (live view). Source: `artifacts/figures/frontend_dashboard_live.png`

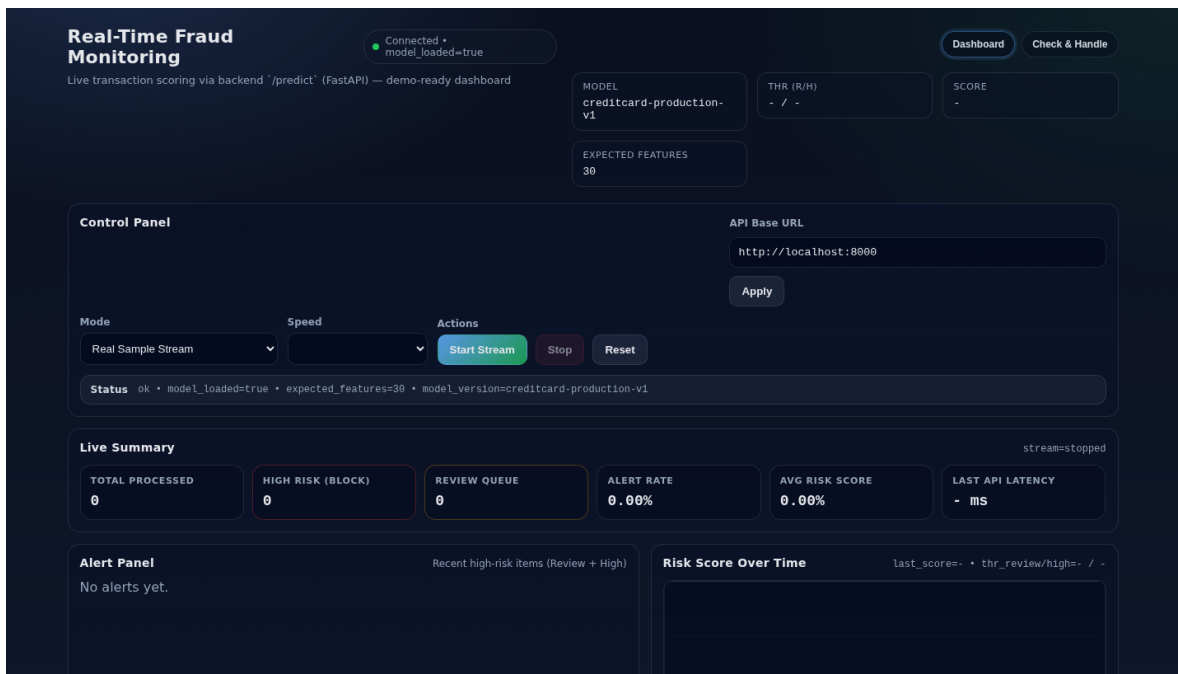


Figure 15: Frontend dashboard (streaming mode). Source: artifacts/figures/frontend-dashboard_streaming.png

```
[LightGBM] [Warning] Stopped training because there are no more leaves that meet the split requirements
/home/and/Documents/Final_Project_DMW501_Group1/.venv/lib/python3.13/site-packages/shap/explainers/_tree.py:620: UserWarning: LightGBM binary classifier with TreeExplainer
r shap values output has changed to a list of ndarray
warnings.warn(
{
  "dataset_path": "data/archive/creditcard.csv",
  "rows": 284807,
  "features": 30,
  "threshold_review": 0.7391262534904675,
  "threshold_high": 0.9999047447104407,
  "selected_model": "logistic regression",
  "artifacts root": "artifacts"
}
(.venv) and@andblap:~/Documents/Final_Project_DMW501_Group1$
(.venv) and@andblap:~/Documents/Final_Project_DMW501_Group1$ docker compose -f deployment/docker-compose.yml up --build
[+] up 24/24
  Image prom/prometheus:v2.54.1 Pulled
  Image grafana/grafana:11.1.0 Pulled
[+] Building 9.7s (12/20)
  => [internal] load local bake definitions
  => => reading from stdin 1.54kB
  => [api internal] load build definition from Dockerfile
  => transferring dockerfile: 421B
  => [frontend internal] load build definition from Dockerfile
  => transferring dockerfile: 300B
  => [mlflow internal] load build definition from Dockerfile
  => transferring dockerfile: 125B
  => [frontend internal] load metadata for docker.io/library/python:3.11-slim
  => [frontend internal] load .dockerignore
  => transferring context: 2B
  => [mlflow internal] load .dockerignore
  => transferring context: 2B
  => [api internal] load .dockerignore
  => transferring context: 2B
  => [frontend 1/5] FROM docker.io/library/python:3.11-slim@sha256:233de06753d30d120b1a3ce359d8d3be8bda78524cd8f520c99883bf33964cf
  => resolve docker.io/library/python:3.11-slim@sha256:233de06753d30d120b1a3ce359d8d3be8bda78524cd8f520c99883bf33964cf
  => sha256:d98f36ba21593a4014243f41ec9c5370abd34f4f65a5d074e41bce23f56a165 14.37MB / 14.37MB
  => sha256:233de06753d30d120b1a3ce359d8d3be8bda78524cd8f520c99883bf33964cf 10.37kB / 10.37kB
  => sha256:a0d516ad25830368043f772cefc1ad733cf77d631c6840b1e9231b0b350 1.73kB / 1.73kB
  => sha256:d1a45c0fb43d72e0c013ee970460a6f49f2c1b0d588b17a558cc6f7878b4276 5.48kB / 5.48kB
  => sha256:5435b2dcdf5cb7fa0d5b1d4d54be2c72a776fab9a605336f5067d6e9ecb5976 29.78MB / 29.78MB
  => sha256:37ffa6577b04e98598e2c10432eb8c66cfc1af6c7698b21c6163f8334606aa14 1.29MB / 1.29MB
  => sha256:0816ee351b60c03c7a77a3f50c717891eedd927f4e6a1d59f6cce05268065606 251B / 251B
  => extracting sha256:5435b2dcdf5cb7fa0d5b1d4d54be2c72a776fab9a605336f5067d6e9ecb5976 1.0s
  => extracting sha256:37ffa6577b04e98598e2c10432eb8c66cfc1af6c7698b21c6163f8334606aa14 0.1s
  => extracting sha256:d98f36ba21593a4014243f41ec9c5370abd34f4f65a5d074e41bce23f56a165 0.6s
  => extracting sha256:0816ee351b60c03c7a77a3f50c717891eedd927f4e6a1d59f6cce05268065606 0.0s
  => [frontend internal] load build context
  => transferring context: 92.31kB
  => [api internal] load build context
  => transferring context: 223.21kB
  => [mlflow 2/5] WORKDIR /app
  => [mlflow 3/3] RUN pip install --no-cache-dir mlflow
  => [api 3/5] RUN apt-get update && apt-get install -y --no-install-recommends curl && rm -rf /var/lib/apt/lists/*
  => # Hit:1 http://deb.debian.org/debian trixie InRelease
  => # Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
  => # Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
  => # Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9671 kB]
```

Figure 16: Docker Compose runtime screenshot. Source: artifacts/deloys/docker-compose.png

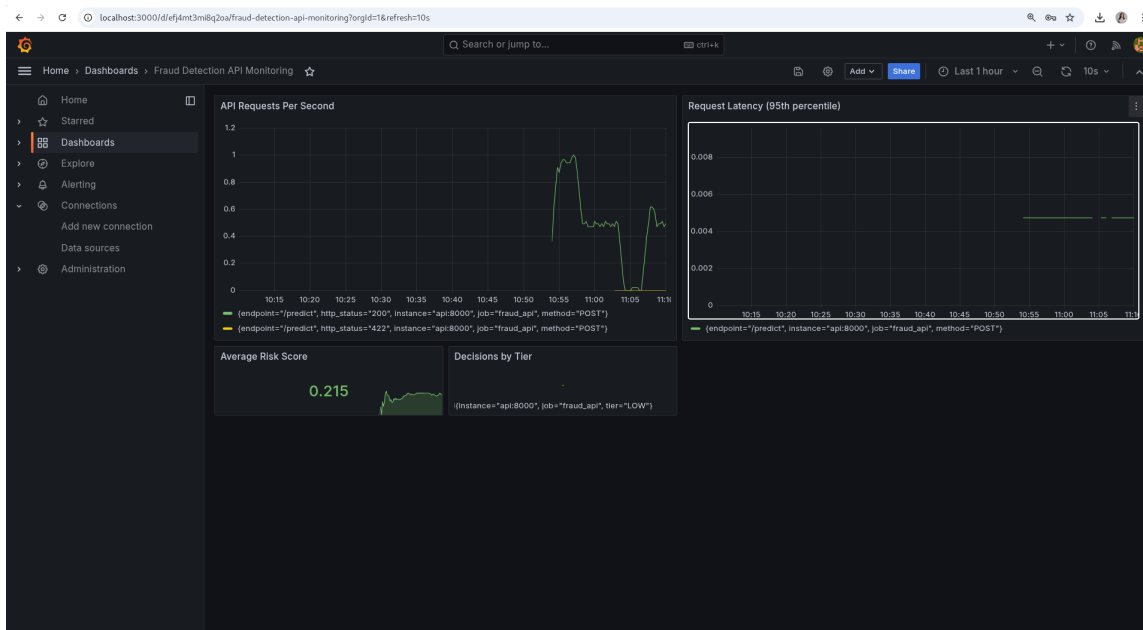


Figure 18: Grafana monitoring dashboard screenshot. Source: artifacts/deploy/Grafana-dashboard.png

The screenshot shows the Prometheus 'Targets' page. It includes a search bar and a table of targets. The table has the following columns: Endpoint, State, Labels, Last Scrape, Scrape Duration, and Error. The table contains one entry for the 'fraud_api' target.

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://api:8000/metrics	UP	instance="api:8000" job="fraud_api"	22.67s ago	1.378ms	

Figure 19: Prometheus targets screenshot. Source: artifacts/deploy/Prometheus-targets.png